Informe Técnico — Sokoban

> **Trabajo Integrador** — Proceso de Desarrollo de Software (PDS)
> **Docente:** Prof. Jonathan Pepe — Fundación UADE
> **Fecha de entrega:** Noviembre 2026

Datos del integrante

Apellido y Nombre	Legajo	Mail
Abdo Ferez, Federico	1194765	fabdoferez@uade.edu.ar

Equipo de desarrollo: Abdo Ferez, Federico — entrega individual (1 integrante).

Índice

1. Introducción
2. Descripción del juego
3. Arquitectura general y MVC
4. Patrones de diseño GoF aplicados
5. Patrones GRASP aplicados
6. Principios SOLID
7. Diseño de niveles
8. Interfaz gráfica (Swing)
9. Suite de pruebas
10. Métricas del proyecto
11. Decisiones de diseño relevantes
12. Uso de herramientas de IA
13. Conclusiones
14. Anexo — Diagramas de clases

1. Introducción

El presente informe documenta el diseño, la implementación y las decisiones técnicas del trabajo integrador de la materia Proceso de Desarrollo de Software: un juego **Sokoban** desarrollado en **Java 21** con interfaz gráfica en **Swing**.

El objetivo del trabajo es aplicar de forma práctica los patrones de diseño estudiados en la materia (GoF, GRASP) y los principios SOLID, demostrando que cada patrón responde a un problema concreto del dominio y no a una imposición teórica.

El proyecto consta de **89 clases de producción** y **22 clases de prueba**, con una suite de **108 tests JUnit 5** que pasan en su totalidad (`BUILD SUCCESS`). El juego incluye 5 niveles con dificultad creciente, 21 sprites PNG, 9 efectos de sonido WAV, niebla de guerra, sistema de puntuación con estrellas y persistencia de progreso.

2. Descripción del juego

2.1 Reglas básicas

Sokoban es un juego de lógica donde el jugador debe empujar cajas hasta sus casillas de destino. El jugador solo empuja (nunca tira), no puede empujar más de una caja a la vez, ni atravesar paredes ni cajas.

2.2 Tipos de cajas

Tipo de caja	Símbolo	Comportamiento
Normal	`$`	Se empuja sin límite
Frágil	`1` – `9`	Soporta N empujes; el siguiente la rompe (derrota)
Llave	`K`	Abre muros al caer sobre el casillero cerrojo correspondiente; solo puede guardarse en su destino después de haber abierto su muro

2.3 Tipos de baldosas

Baldosa	Símbolo	Efecto
Piso	` `	Sin efecto
Pared	`#`	Bloquea jugador y cajas, no deja ver
Vacío	`%`	Fuera del tablero, bloquea todo
Hielo	`~`	Desliza al ocupante hasta la siguiente baldosa que no sea hielo
Portal	`P` – `S`	Teletransporta al ocupante al portal pareado
Destino	`.` `:` `;`	Casilla donde guardar cajas (normal, frágil, llave). Vacío bloquea al jugador (es un «agujero»); lleno hace de puente
Casillero cerrojo	`a` – `e`	Al recibir la caja llave, abre todos los muros de la misma letra
Muro cerrado	`A` – `E`	Bloquea como pared hasta que la llave del grupo lo abre

2.4 Sistema de deshacer

Se permite deshacer los **últimos 15 movimientos**. Cada uso del botón restaura el estado del nivel **5 movimientos** hacia atrás, con un máximo de **3 usos consecutivos**. Al volver a mover, el contador de usos se reinicia.

2.5 Condiciones de victoria y derrota

- **Victoria:** todas las casillas de destino tienen una caja del tipo correcto guardada.

- **Derrota por caja rota** (`BROKEN`): una caja frágil recibe un empuje de más.
- **Derrota por guardado incorrecto** (`WRONG_STORAGE`): una caja queda guardada en un destino del tipo equivocado.
- **Derrota por llave prematura** (`EARLY_KEY_STORAGE`): una caja llave se guarda sin haber abierto su muro.
- **Derrota por deadlock** (`DEADLOCK`): la caja empujada queda trabada en una esquina o un bloque 2×2 irrecuperable. Tiene arreglo con deshacer.
- **Derrota por tiempo** (`TIME_OUT`): se agota el tiempo límite del nivel.

2.6 Puntaje y estrellas

- +1 punto por segundo restante
- +100 puntos por cada caja guardada
- +100 puntos por cada muro abierto
- −1 punto por cada empuje
- −250 puntos por cada uso del botón deshacer
- Estrellas: ≤50% del objetivo → 1★, ≤85% → 2★, >85% → 3★

3. Arquitectura general y MVC

3.1 Estructura de paquetes

```
src/main/java/com/sokoban/
├── Main.java                # punto de entrada (arma las 3 capas en el EDT)
├── model/                   # MODELO — reglas del juego, sin dependencias de UI
│   ├── board/               #   tablero, celdas, posiciones, direcciones
│   ├── box/                 #   jugador y jerarquía de cajas (Template Method)
│   ├── tile/                #   jerarquía de baldosas y efectos (Strategy)
│   │   ├── effect/          #     estrategias de efecto al entrar
│   │   └── mechanism/       #     cerrojos y muros (Observer)
│   ├── game/                #   Level, Game, estados, derrotas, puntaje, eventos
│   │   ├── state/           #     patrón State
│   │   ├── event/           #     patrón Observer (eventos)
│   │   ├── loss/            #     patrón Strategy (derrotas)
│   │   └── score/           #     patrón Strategy (puntaje)
│   └── history/             #   deshacer (Command + Memento)
├── view/                    # VISTA — Swing
├── controller/              # CONTROLADOR — teclado, HUD, reloj
├── factory/                 # carga de niveles (Factory Method + Builder)
├── port/                    # interfaces de salida (DIP)
├── service/                 # adaptadores: sonido, persistencia
└── exception/
```

3.2 Aislamiento MVC

El aislamiento entre capas es estricto:

- **Ninguna clase del paquete** `model/` importa `javax.swing.*` ni `java.awt.*`.
- La **vista** se entera de los cambios del modelo exclusivamente a través del patrón **Observer** (`GameListener`). La vista nunca modifica el modelo: solo lee y repinta.
- Los **controladores** reciben los eventos de Swing (pulsaciones de teclado, clics en botones) y los traducen en llamadas a la fachada `Game`. Nunca acceden a `Board`, `Cell` o `UndoHistory` directamente.
- El **único punto de cruce** entre las tres capas es la clase `Main`, que arma el modelo, construye la vista, crea los controladores y los conecta dentro del Event Dispatch Thread (EDT) de Swing.

4. Patrones de diseño GoF aplicados

4.1 Factory Method (Creacional)

> *«Define una interfaz para crear un objeto, pero deja que las subclases decidan qué clase instanciar.»* — Gamma, Helm, Johnson, Vlissides: *Design Patterns* (1994), p. 107.

Problema: el archivo de texto del nivel usa caracteres (`#`, `$`, `~`, `P`, etc.) para representar las celdas. La lógica de parseo no debería conocer cada tipo de baldosa ni de caja.

Solución:

Rol GoF	Clase
Creator (interfaz)	`CellCreator` — método `create(Position, char)`
ConcreteCreator	`WallCreator`, `DoorCreator`, `KeyTileCreator`, `PortalCreator`, `WeakBoxCreator`
Product	`CellContent` — record inmutable que agrupa `Tile` + `Occupant`

En `LevelLoader`, un `Map<Character, CellCreator>` asocia cada carácter a su fábrica. Agregar un nuevo tipo de baldosa al juego consiste en implementar un `CellCreator` y registrarlo en el mapa, sin modificar el parser (**OCP**).

4.2 Builder (Creacional)

> *«Separa la construcción de un objeto complejo de su representación.»* — Gamma et al., p. 97.

Problema: construir un `Level` requiere validar 5 reglas de consistencia complejas: balance de cajas vs. destinos por tipo, apareamiento de portales, correspondencia cerrojo ↔ muro, cierre perimetral y jugador único.

Solución:

Rol GoF	Clase
Director	`LevelLoader` — lee el archivo de texto y dirige la construcción
Builder	`LevelBuilder` — acumula las celdas con `withHeader()` → `addCell()` → `build()`

La validación y el cableado bidireccional de portales y grupos cerrojo/muro ocurren en el paso final `build()`, garantizando que ninguna instancia inválida de `Level` llegue a existir. Si alguna regla falla, se lanza `InvalidLevelException` con el número de línea del archivo.

4.3 Template Method (Comportamiento)

> *«Define el esqueleto de un algoritmo en una operación, difiriendo algunos pasos a las subclases.»* — Gamma et al., p. 325.

Problema: cada tipo de caja reacciona distinto al ser empujada (`WeakBox` se daña, `NormalBox` no), pero el esqueleto del empuje y del guardado es siempre el mismo.

Solución:

```
// Box.java — clase abstracta
public final PushResult push() {
    return onPushed();   // hook
}
public final StorageResult storeAt(StoragePoint point) {
    return onStored(point);  // hook
}

// WeakBox.java — redefine el hook
@Override
protected PushResult onPushed() {
    if (remainingPushes == 0) return PushResult.BROKEN;
    remainingPushes--;
    return PushResult.PUSHED;
}

// KeyBox.java — redefine el hook
@Override
protected StorageResult onStored(StoragePoint point) {
    return doorOpened ? StorageResult.STORED : StorageResult.KEY_NOT_USED;
}
```

Los métodos `push()` y `storeAt()` son `final`: las subclases solo redefinen los hooks. El tablero llama a `box.push()` sin conocer el tipo concreto de caja.

4.4 State (Comportamiento)

> *«Permite a un objeto alterar su comportamiento cuando su estado interno cambia. El objeto parecerá haber cambiado de clase.»* — Gamma et al., p. 305.

Problema: el juego tiene 6 fases distintas (menú, jugando, nivel ganado, nivel perdido, deadlock con arreglo, juego completo). Cada fase tiene reglas distintas sobre qué teclas aceptar y qué transiciones permitir.

Solución:

Rol GoF	Clase
State (interfaz)	`GameState` — `acceptsInput()`, `onMove()`, `onUndo()`, `onNextLevel()`
ConcreteState ×6	`MenuState`, `PlayingState`, `WonState`, `LostState`, `BlockedState`, `GameCompletedState`

Rol GoF	Clase
Context	`Game`

`Game` **no tiene ni un solo** `if` **o** `switch` **sobre el estado**. Delega cada acción al estado actual y reemplaza su estado por el que le devuelven:

```
// Game.java
public void move(Direction d) { state = state.onMove(this, d); }
public void undo()            { state = state.onUndo(this); }
public void nextLevel()       { state = state.onNextLevel(this); }
```

El bloqueo del teclado no es un booleano suelto: es `state.acceptsInput()`. Cada estado concreto solo escribe las transiciones que le corresponden; los métodos sin transición heredan la implementación vacía de la interfaz (default methods).

Ejemplo de transición en `PlayingState`:

```
public GameState onMove(Game game, Direction direction) {
    game.applyMove(direction);
    Level level = game.getCurrentLevel();
    if (level.isWon()) return new WonState();
    LossReason reason = level.getLossReason();
    if (reason == null) return this;
    return reason == LossReason.DEADLOCK ? new BlockedState() : new LostState();
}
```

El deadlock pasa a `BlockedState` (donde `onUndo` devuelve a `PlayingState`), mientras que las demás derrotas pasan a `LostState` (donde el input está completamente bloqueado).

4.5 Observer (Comportamiento) — Doble aplicación

> *«Define una dependencia uno-a-muchos entre objetos para que cuando un objeto cambie de estado, todos sus dependientes sean notificados automáticamente.»* — Gamma et al., p. 293.

4.5.1 Eventos de partida

Problema: la vista, el sonido, la niebla de guerra y la persistencia de progreso necesitan reaccionar ante cada acción del juego, pero el modelo no puede conocer Swing ni la capa de servicios.

Solución:

Rol GoF	Clase
Subject	`Level` — `addListener(GameListener)`, `publish(GameEvent)`
Observer (interfaz)	`GameListener` — `onGameEvent(GameEvent)`
ConcreteObserver	`BoardView` (repinta), `HudView` (actualiza contadores), `SoundService` (reproduce audio), `ProgressSaver` (persiste al ganar)

Rol GoF	Clase
Mensaje	`GameEvent` — contiene `EventType`, `Position` y `Direction`

Los `EventType` son: `MOVE`, `PUSH`, `STORE`, `BREAK`, `DOOR_OPENED`, `TELEPORT`, `WIN`, `LOSE`, `UNDO`.

Reiniciar o pasar de nivel construye un `Level` nuevo. Para que la vista y el sonido no queden suscriptos al nivel viejo, `Game.addLevelListener()` registra los listeners y los reengancha automáticamente a cada nivel que se carga.

4.5.2 Mecanismo cerrojo ↔ puerta

Problema: cuando una caja llave cae sobre un casillero cerrojo, deben abrirse todos los muros de la misma letra. Pero `KeyTile` y `Door` no deben conocerse directamente para mantener bajo acoplamiento.

Solución:

Rol GoF	Clase
Subject	`KeyTile` — `addListener(KeyTileListener)`
Observer (interfaz)	`KeyTileListener` — `onKeyTileActivated(char group)`
ConcreteObserver	`DoorGroup` — agrupa todos los `Door` de una letra y los abre de una vez

```
// KeyTile.java — notifica
public LandingOutcome receiveBox(Box box) {
    if (!box.opensDoors()) return LandingOutcome.NONE;
    box.markDoorOpened();
    for (KeyTileListener listener : listeners) {
        listener.onKeyTileActivated(group);
    }
    return LandingOutcome.DOOR_OPENED;
}

// DoorGroup.java — escucha y actúa
public void onKeyTileActivated(char activatedGroup) {
    if (activatedGroup == group) open();
}
```

`LevelBuilder.wireDoorGroups()` arma los grupos al construir el nivel y suscribe cada `DoorGroup` al `KeyTile` de su letra. `Door` y `KeyTile` nunca se ven directamente: se hablan a través del grupo.

4.6 Strategy (Comportamiento) — Triple aplicación

> *«Define una familia de algoritmos, encapsula cada uno de ellos y los hace intercambiables.»* — Gamma et al., p. 315.

4.6.1 Efectos de entrada al terreno

Rol GoF	Clase
Strategy	`EntryEffect` — `resolve(Board, Occupant, Position, Direction): Position`
ConcreteStrategy ×3	`NoEffect` (devuelve la posición tal cual), `SlideEffect` (hielo: avanza recursivamente), `TeleportEffect` (portal: posición de la salida)
Context	`Board` — aplica `tile.getEntryEffect().resolve(...)` tras cada entrada

Cada baldosa expone su efecto con `getEntryEffect()`. El hielo devuelve un `SlideEffect` compartido; el piso devuelve una única instancia compartida de `NoEffect`. El `SlideEffect` delega en el efecto de la celda siguiente en lugar de iterar: si esa celda es hielo sigue deslizando, si es piso el `NoEffect` corta la recursión.

Decisión de diseño: el efecto es **del terreno**, no de la caja. Por eso vive en `Tile` y no en `Box`: si se agrega un nuevo terreno (lava, cintas transportadoras), se implementa un nuevo `EntryEffect` sin tocar la jerarquía de cajas.

4.6.2 Condiciones de derrota

Rol GoF	Clase
Strategy	`LossCondition` — `isMet(Level): boolean`, `getReason(): LossReason`
ConcreteStrategy ×5	`BrokenBoxCondition`, `WrongStorageCondition`, `EarlyKeyStorageCondition`, `DeadlockCondition`, `TimeOutCondition`

`Level` mantiene una lista estática de las 5 condiciones y las evalúa en orden después de cada movimiento. La victoria se chequea **antes** que el deadlock, para que una caja que se guarda en la esquina correcta no se considere trabada.

Agregar una nueva condición de derrota es crear una clase que implemente `LossCondition` y agregarla a la lista, sin modificar `Level` (**OCP**).

4.6.3 Cálculo de puntaje

Rol GoF	Clase
Strategy	`ScoreCalculator` — `calculate(...): ScoreResult`
ConcreteStrategy	`StandardScoreCalculator`
Resultado	`ScoreResult` — `score` + `stars`

La fórmula se concentra en una clase aislada, fácilmente reemplazable si se quisiera cambiar el criterio de puntuación.

4.7 Memento (Comportamiento)

> *«Sin violar la encapsulación, captura y externaliza el estado interno de un objeto para que pueda ser*

> *restaurado más tarde.»* — Gamma et al., p. 283.

Problema: con hielo, portales, puertas que se abren irreversiblemente y casilleros cerrojo que se consumen, **invertir un movimiento es irresoluble**. Un solo empuje puede mover una caja cuatro celdas (hielo), teletransportarla (portal), abrir tres muros y consumir una baldosa.

Solución:

Rol GoF	Clase
Originator	`Level` — `saveToMemento()`, `restoreFrom(LevelMemento)`
Memento	`LevelMemento` — foto inmutable del tablero (posición del jugador, snapshots de celdas, estado de puertas abiertas/cerradas, cajas guardadas en destinos, estado interno de cada caja)
Caretaker	`UndoHistory` — cola circular de 15 fotos, 5 pasos por uso, 3 usos consecutivos

`LevelMemento` guarda **referencias** a los objetos del tablero junto con su estado mutable. Al restaurar, vuelven las mismas cajas y baldosas (no copias), y se les setea su estado anterior.

`BoxSnapshot` captura el estado interno de cada caja (empujes restantes de `WeakBox`, llave usada de `KeyBox`) para restaurarlo sin castear:

```
public record BoxSnapshot(Box box, int remainingPushes, boolean doorOpened) {
    public static BoxSnapshot of(Box box) {
        return new BoxSnapshot(box, box.getRemainingPushes(), box.hasOpenedDoor());
    }
    public void restore() {
        box.setRemainingPushes(remainingPushes);
        box.setDoorOpened(doorOpened);
    }
}
```

Los contadores de `LevelStats` (movimientos, empujes, undos) **no se restauran** a propósito: los movimientos realmente ocurrieron y el puntaje los penaliza.

4.8 Command (Comportamiento)

> *«Encapsula una petición como un objeto, permitiendo parametrizar clientes con diferentes peticiones.»* — Gamma et al., p. 233.

Solución:

Rol GoF	Clase
Command (interfaz)	`Command` — `execute()`
ConcreteCommand	`MoveCommand` — encapsula la dirección y la referencia al `Game`
Invoker	`InputController` — enlaza cada tecla direccional a un `MoveCommand` en el `ActionMap` de Swing

```
// InputController.java
keys.forEach((keyName, direction) -> {
    String id = "move_" + keyName;
    inputMap.put(KeyStroke.getKeyStroke(keyName), id);
    actionMap.put(id, run(new MoveCommand(game, direction)));
});
```

Se usa `InputMap` / `ActionMap` en lugar de `KeyListener` para no depender de qué componente tiene el foco dentro de la ventana.

4.9 Facade / Fachada (Estructural)

> *«Proporciona una interfaz unificada para un conjunto de interfaces de un subsistema.»* — Gamma et al., p. 185.

Solución:

- `Game` es la **única fachada del modelo** hacia el controlador. `InputController` y `HudController` solo hablan con `Game`, nunca con `Board`, `Cell` o `UndoHistory`.
- `GameWindow` actúa como fachada de la vista Swing, gestionando el `CardLayout` entre `MenuView`, `BoardView` y `SummaryView`.

4.10 Adapter (Estructural)

> *«Convierte la interfaz de una clase en otra interfaz que es la que esperan los clientes.»* — Gamma et al., p. 139.

Problema: el juego necesita guardar el progreso del jugador entre sesiones, pero el modelo no debería depender del sistema de archivos ni de `java.nio`.

Solución:

Rol GoF	Clase
Target (la interfaz que espera el cliente)	`ProgressRepository` — `save(Progress)`, `load(): Progress`
Adapter	`FileProgressRepository` — implementa `ProgressRepository` traduciendo cada llamada a lecturas y escrituras de un archivo de texto
Adaptee	La API de archivos de Java (`java.nio.file.Files`)
Client	`ProgressSaver` y el menú, que solo conocen `ProgressRepository`

El modelo depende de la abstracción `ProgressRepository` (principio de inversión de dependencias, ver 6.5); la implementación concreta en disco vive en la capa de servicios. Cambiar el archivo por otra forma de guardado es escribir otro adaptador, sin tocar el modelo.

5. Patrones GRASP aplicados

Los patrones GRASP (Craig Larman, *Applying UML and Patterns*, 2004) se aplican en las siguientes ubicaciones:

Patrón GRASP	Dónde se aplica	Evidencia
Information Expert	`Board`, `Box`, `Tile`, `LevelStats`	El tablero conoce todas las celdas → valida y ejecuta movimientos. La caja sabe si aguanta el empuje. La baldosa sabe a quién deja pasar. Los contadores viven con quien los conoce.
Creator	`LevelBuilder`	Crea `Cell[][]`, `Board` y `Level` porque agrega sus partes.
Controller	`InputController`, `HudController`	Reciben los eventos del sistema (Swing) y delegan en la fachada `Game`.
Low Coupling	Paquetes `model` / `view` / `controller` / `port`	El modelo no importa nada de UI. La vista no modifica el modelo.
High Cohesion	Un paquete por concepto	`board`, `box`, `tile`, `history`, `loss`, `score`, `state` — cada uno tiene una sola razón de cambio.
Polymorphism	Jerarquía de `Tile` (8 subclases) y `Box` (3 subclases); `Occupant.canEnter(Tile)`	No hay un solo `switch` sobre tipo de baldosa ni de caja en el motor de juego (ver 5.1).
Pure Fabrication	`LevelLoader`, `UndoHistory`, `SoundService`, `GameClock`, `FileProgressRepository`, `ScoreCalculator`, `SpriteCatalog`	Clases que no representan conceptos del dominio real del Sokoban, creadas para bajar acoplamiento y subir cohesión.
Indirection	`DoorGroup`, `ProgressRepository`, `LevelLoader.creators`	Objetos intermediarios que evitan el acoplamiento directo. `KeyTile` y `Door` no se conocen entre sí: se hablan a través del grupo.
Protected Variations	`CellCreator`, `LossCondition`, `EntryEffect`, `ScoreCalculator`, `ProgressRepository`	Puntos de variación aislados detrás de interfaces.

5.1 Polymorphism: quién puede entrar a una baldosa

Problema: el tablero necesita saber si un ocupante puede entrar a una baldosa, pero la regla depende de **ambos**: el tipo de ocupante y el tipo de baldosa. El jugador pregunta si puede pisar; la caja pregunta si puede ser empujada ahí.

Solución: en lugar de que `Board` pregunte «¿sos caja o jugador?», cada ocupante resuelve su caso por polimorfismo y le hace a la baldosa la pregunta que le corresponde:

```
// Occupant.java — interfaz
boolean canEnter(Tile tile);

// Player.java
public boolean canEnter(Tile tile) { return tile.allowsPlayer(); }

// Box.java
public boolean canEnter(Tile tile) { return tile.allowsBox(this); }
```

El tablero llama `occupant.canEnter(tile)` sin un solo `instanceof`. La primera llamada se resuelve según el tipo de ocupante y la segunda según el tipo de baldosa, así que agregar una baldosa nueva solo implica redefinir `allowsPlayer()` y `allowsBox()`.

5.2 Pure Fabrication: el catálogo de sprites

`SpriteCatalog` no representa nada del dominio del Sokoban: es una clase de la vista creada para que `BoardView` no cargue imágenes. Guarda un `Map<SpriteKey, BufferedImage>` y lee cada PNG del disco **una sola vez**, la primera vez que se pide. El `BoardView` dibuja hasta 9×13 = 117 celdas en cada repintado, y todas las celdas de piso (o pared, etc.) reciben la misma imagen ya cargada. `SoundService` aplica el mismo criterio con los 9 efectos WAV, precargados como `Clip` indexados por `SoundEffect`.

6. Principios SOLID

6.1 Single Responsibility (SRP)

Cada clase tiene una única responsabilidad bien definida:

- `UndoHistory` solo gestiona snapshots.
- `StandardScoreCalculator` solo calcula puntajes.
- `VisibilityMap` solo calcula el BFS de celdas visibles.

Nota de diseño: `Level` tiende a acumular responsabilidades (mover, deshacer, victoria, derrota, tiempo, eventos, visibilidad). Se mitiga delegando cada una a un colaborador: `UndoHistory`, `LevelMemento`, la lista de `LossCondition`, `VisibilityMap` y `LevelStats`.

6.2 Open/Closed (OCP)

El código está abierto a la extensión y cerrado a la modificación:

- Nuevo tipo de baldosa → implementar un `CellCreator` y registrarlo en `LevelLoader`, sin tocar el parser.
- Nueva condición de derrota → implementar `LossCondition` y agregarla a la lista.
- Nuevo efecto de terreno → implementar `EntryEffect`.

6.3 Liskov Substitution (LSP)

Las subclases de `Tile` y `Box` pueden sustituirse en cualquier celda del `Board` sin violar invariantes. Las subclases solo restringen vía booleanos (`allowsPlayer()`, `allowsBox()`), nunca lanzan excepciones nuevas ni cambian el contrato de los métodos.

6.4 Interface Segregation (ISP)

Todas las interfaces del proyecto son mínimas:

- `Occupant`: 2 métodos (`isPushable`, `canEnter`).
- `Command`: 1 método (`execute`).
- `GameListener`: 1 método (`onGameEvent`).
- `CellCreator`: 1 método (`create`).
- `LossCondition`: 2 métodos (`isMet`, `getReason`).
- `ScoreCalculator`: 1 método (`calculate`).
- `ProgressRepository`: 2 métodos (`save`, `load`).
- `KeyTileListener`: 1 método (`onKeyTileActivated`).

6.5 Dependency Inversion (DIP)

Los módulos de alto nivel dependen de abstracciones:

- `Game` depende de `IntFunction<Level>`, no de `LevelLoader`.
- El modelo depende del puerto `ProgressRepository`; la implementación concreta `FileProgressRepository` vive en la capa de servicios.

7. Diseño de niveles

El juego incluye 5 niveles con dificultad creciente. Cada uno introduce un mecanismo nuevo:

Nivel	Nombre	Tamaño	Introduce	Empujes
01	Primeros pasos	7×8	Cajas normales + destinos (tutorial)	6
02	Carga frágil	8×10	Cajas frágiles con los empujes justos	10
03	Pista de hielo	9×11	Terreno resbaladizo para jugador y cajas	4
04	Bajo llave	9×11	Caja llave + casillero cerrojo + muro + **niebla de guerra**	11
05	El almacén final	9×13	Portales + los tres tipos de caja + muro	12

Los niveles están definidos en archivos de texto (`src/main/resources/levels/nivel01.txt` a `nivel05.txt`) y las soluciones están documentadas en `docs/soluciones-niveles.md` . El test de integración `LevelSolutionTest` reproduce las 5 soluciones automáticamente y verifica que el juego llegue a `GameCompletedState` .

Nota sobre la niebla de guerra: el showcase real es el **nivel 04**. La mitad inferior del mapa solo es alcanzable cruzando el muro `A` , así que arranca oculta y se revela al abrirlo. En el nivel 05, los portales dejan ver la zona de abajo desde el arranque (la visibilidad atraviesa portales), así que la niebla no esconde nada.

8. Interfaz gráfica (Swing)

8.1 Arquitectura de la vista

```
JFrame
└── GameWindow                          (CardLayout: MENU | GAME | SUMMARY)
    ├── MenuView          JPanel        → título, "Jugar", "Continuar", "Salir"
    ├── gamePanel          JPanel(BorderLayout)
    │   ├── HudView       JPanel NORTH → nivel, movimientos, empujes, tiempo,
    │   │                                undos restantes, [Deshacer] [Reiniciar]
    │   └── BoardView     JPanel CENTER→ paintComponent() dibuja la grilla
    └── SummaryView       JPanel        → resumen de fin de nivel con puntaje y estrellas
```

8.2 Dibujo del tablero

- Tamaño de celda: **48 px** fijos.
- Capas de dibujo: 1) baldosa (piso/pared/hielo/destino/cerrojo/muro) → 2) ocupante (caja o jugador) → 3) velo de niebla (rectángulo negro con `AlphaComposite` al 85%).
- Antialiasing: `NEAREST_NEIGHBOR` (es pixel art).
- El sprite del jugador cambia según `Player.getFacing()`.
- Las cajas guardadas usan un sprite distinto al de la caja suelta.

8.3 Controladores

Clase	Responsabilidad	Patrón
`InputController`	Traduce ← ↑ → ↓ a `Direction` y llama a `game.move(d)` vía `Command`	Controller (GRASP) + Command (GoF)
`HudController`	`ActionListener` de los botones Deshacer y Reiniciar	Controller
`GameClock`	`javax.swing.Timer` de 1 segundo que llama a `level.elapseSeconds(1)`	Pure Fabrication

8.4 Sonido

`SoundService` usa `javax.sound.sampled.Clip` (no JavaFX). Es `GameListener`: mapea cada `EventType` a un `SoundEffect`. Cada `Clip` se precarga una vez y se reutiliza. Los archivos son WAV PCM 16-bit, que es lo que `javax.sound.sampled` soporta nativamente.

8.5 Assets

- **21 sprites PNG** de 48×48 pixels: piso, pared, vacío, hielo, portal, 3 tipos de caja (normal + guardada), 3 tipos de destino, casillero cerrojo, muro abierto/cerrado, jugador en 4 direcciones.
- **9 efectos de sonido WAV**: movimiento, empuje, guardado, rotura, apertura de puerta, teletransporte, victoria, derrota, deshacer.

- Todos los assets son **obra propia** generados programáticamente con `Graphics2D` y `javax.sound.sampled`, por lo que cumplen el requisito de **dominio público / CC0** sin depender de terceros. Están acreditados en `src/main/resources/assets/CREDITS.md`.

9. Suite de pruebas

El proyecto incluye **108 tests JUnit 5** organizados en 21 clases de prueba:

Clase de test	Tests	Qué verifica
`LevelLoaderTest`	11	Parseo de niveles, validaciones del builder, carga real de los 5 niveles
`LevelSolutionTest`	2	Juega los 5 niveles con las soluciones documentadas y verifica `GameCompletedState`
`MovementTest`	7	Movimiento del jugador, colisiones, puentes sobre destinos
`PositionTest`	4	Aritmética de posiciones y direcciones
`PushTest`	7	Empuje de cajas, bloqueos, cajas en hielo
`VisibilityMapTest`	4	BFS de visibilidad, paredes en diagonal, portales
`NormalBoxTest`	3	Empuje sin límite, guardado
`WeakBoxTest`	4	Resistencia, rotura, guardado
`GameEventTest`	5	Emisión y recepción de eventos del Observer
`LevelStatsTest`	2	Contadores de movimientos, empujes y undos
`LossConditionTest`	7	Las 5 condiciones de derrota, orden de prioridad
`ScoreCalculatorTest`	5	Fórmula de puntaje y umbrales de estrellas
`GameStateTest`	7	Transiciones del patrón State
`WinConditionTest`	5	Victoria polimórfica, input bloqueado tras ganar
`UndoTest`	10	Ventana de 15, pasos de 5, 3 usos consecutivos, reinicio tras mover
`IceTest`	5	Deslizamiento de jugador y cajas, parada contra obstáculo
`DoorKeyTileTest`	7	Apertura de muros, consumo de cerrojo, llave prematura
`PortalTest`	5	Teletransporte en ambos sentidos, portal bloqueado
`StoragePointTest`	5	Guardado correcto/incorrecto, puente, victoria
`ProgressRepositoryTest`	2	Persistencia y carga de progreso desde archivo
`GameFlowTest`	1	Flujo completo de UI: jugar 5 niveles y volver al menú

Todos los tests pasan: `Tests run: 108, Failures: 0, Errors: 0, Skipped: 0 → BUILD SUCCESS`.

10. Métricas del proyecto

Métrica	Valor
Clases de producción (`.java` en `src/main`)	89
Clases de prueba (`.java` en `src/test`)	22
Líneas de producción	3.177
Líneas de test	1.719
Tests JUnit 5	108 (todos en verde)
Niveles jugables	5
Sprites PNG	21 (48×48 px)
Efectos de sonido WAV	9
Patrones GoF aplicados	10 (Factory Method, Builder, Template Method, State, Observer ×2, Strategy ×3, Memento, Command, Facade, Adapter)
Versión de Java	21
Framework gráfico	Swing (`javax.swing.*`)
Build system	Maven 3 (wrapper incluido)
Repositorio	GitHub — `https://github.com/FedeAbdo15/tpo-sokoban.git`

11. Decisiones de diseño relevantes

11.1 Memento vs. Undo inverso

Se eligió **Memento** (restaurar una foto del tablero) en lugar de deshacer invirtiendo el último movimiento. Con hielo, portales, puertas que se abren y baldosas que se consumen, un solo empuje puede encadenar efectos irreversibles. Restaurar una foto es correcto por construcción y no depende de cuántos efectos se encadenaron.

11.2 Destino vacío como agujero

Un destino vacío (`.` , `:` , `;`) bloquea al jugador. Cuando recibe la caja correcta, la guarda internamente y la celda queda libre: el jugador puede caminar por encima (la caja guardada hace de «puente»). Una caja guardada **no se puede volver a empujar**.

Esta decisión elimina toda ambigüedad sobre la coexistencia de jugador y caja en la misma celda (la `Cell` solo tiene un lugar de ocupante) y hace que la victoria sea trivial y polimórfica: `Board.isComplete()` recorre las celdas preguntando `tile.isPendingStorage()` sin un solo `instanceof` .

11.3 Consecutividad del deshacer

La consigna dice «máximo de tres usos consecutivos». Se eligió la lectura literal: se pueden encadenar 3 usos, y **al volver a mover el contador se reinicia**, porque la ventana de 15 movimientos se desliza y vuelve a haber historial disponible. Si en la defensa se prefiere la lectura alternativa (3 usos totales por nivel), el cambio es una línea en `UndoHistory` (no reiniciar el contador en `record()`).

11.4 Migración de JavaFX a Swing

El proyecto originalmente usaba el arquetipo JavaFX de IntelliJ. Se migró a Swing antes de escribir una sola línea de UI propia: se eliminaron las dependencias de `javafx-controls` , `javafx-fxml` y `javafx-media` del `pom.xml` , se borró el paquete scaffolding y se escribió la interfaz en Swing desde cero. El costo fue cero porque el modelo ya estaba aislado de la UI.

11.5 InputMap/ActionMap vs. KeyListener

Un `KeyListener` depende de qué componente tiene el foco. Al tocar un botón del HUD, las flechas dejaban de funcionar. `InputMap` / `ActionMap` con `WHEN_IN_FOCUSED_WINDOW` resuelve el problema. Los botones del HUD además tienen `setFocusable(false)` .

12. Uso de herramientas de IA

Se declara, conforme a la consigna, el uso de herramientas de inteligencia artificial como apoyo durante el desarrollo. En particular:

- **Generación de código asistida:** se utilizaron modelos de lenguaje como apoyo para la implementación de clases, particularmente en la capa de vista (Swing) y en la generación programática de sprites y efectos de sonido.
- **Análisis y documentación:** se usaron para analizar la estructura del proyecto, verificar la correcta aplicación de patrones y redactar documentación técnica.
- **Revisión y validación:** todo el código generado fue **revisado, comprendido y validado** por el autor. Se verificó que cada patrón estuviera correctamente aplicado consultando la bibliografía de la materia (Gamma et al., Larman, Martin).

El alcance concreto: las herramientas de IA asistieron en la escritura y refactoreo del código, pero todas las **decisiones de diseño** (elección de patrones, estructura de paquetes, reglas del juego, diseño de niveles y resolución de ambigüedades con la consigna) fueron tomadas por el autor.

13. Conclusiones

El proyecto Sokoban demuestra la aplicación práctica de **10 patrones GoF**, **9 patrones GRASP** y los **5 principios SOLID** en un sistema de complejidad real. Cada patrón responde a un problema concreto del dominio:

1. **State** elimina la lógica condicional dispersa sobre las fases del juego.
2. **Memento** resuelve la imposibilidad matemática de deshacer movimientos con efectos encadenados.
3. **Strategy** (triple) desacopla los algoritmos de terreno, derrota y puntaje del código que los invoca.
4. **Observer** (doble) permite que la vista, el sonido y la persistencia reaccionen sin que el modelo los conozca.
5. **Template Method** centraliza el esqueleto del empuje y deja que cada tipo de caja solo redefina su regla.
6. **Factory Method + Builder** aíslan el formato del archivo del modelo de dominio.

El resultado es un juego completo y jugable, con 108 tests en verde, 5 niveles con dificultad creciente, interfaz gráfica en Swing con sprites y sonido, y un modelo que no importa una sola clase de la UI.

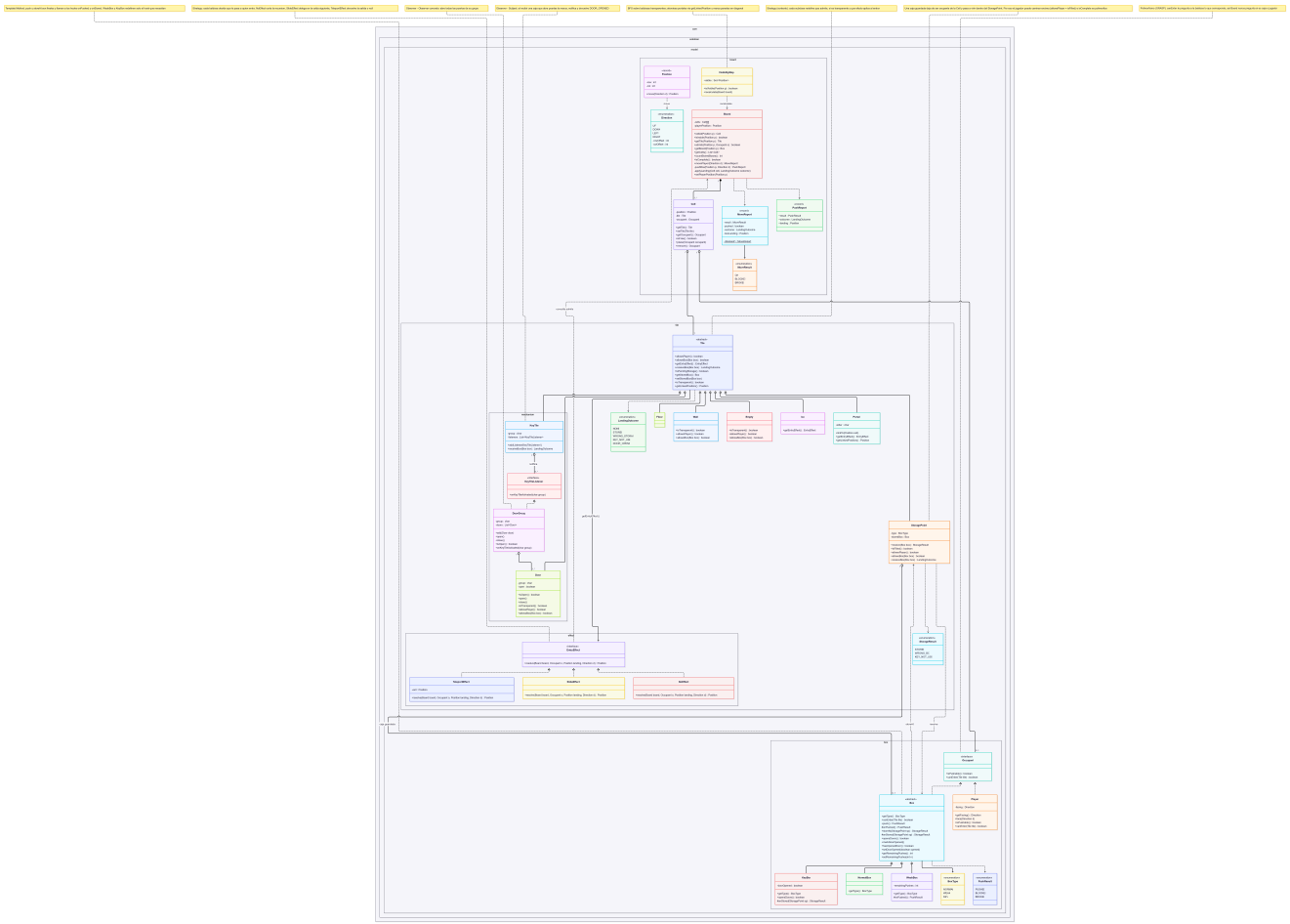
Bibliografía

- Gamma, E., Helm, R., Johnson, R. y Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Larman, C. (2004). *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development* (3ra ed.). Prentice Hall.
- Martin, R. C. (2002). *Agile Software Development: Principles, Patterns, and Practices*. Prentice Hall.
- Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.

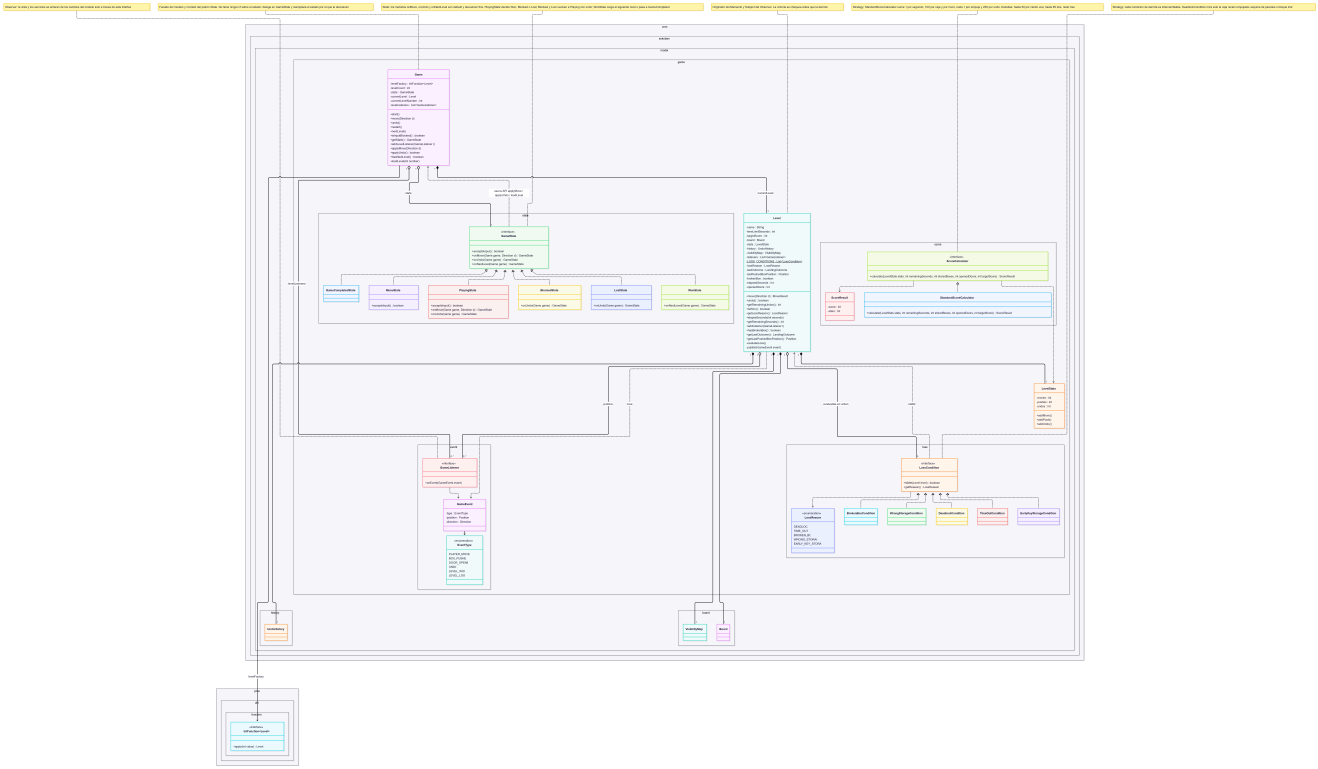
Anexo — Diagramas de clases

El diagrama de clases está dividido en cuatro vistas. La fuente de cada una (Mermaid) y su imagen están en `documentation/umls/`.

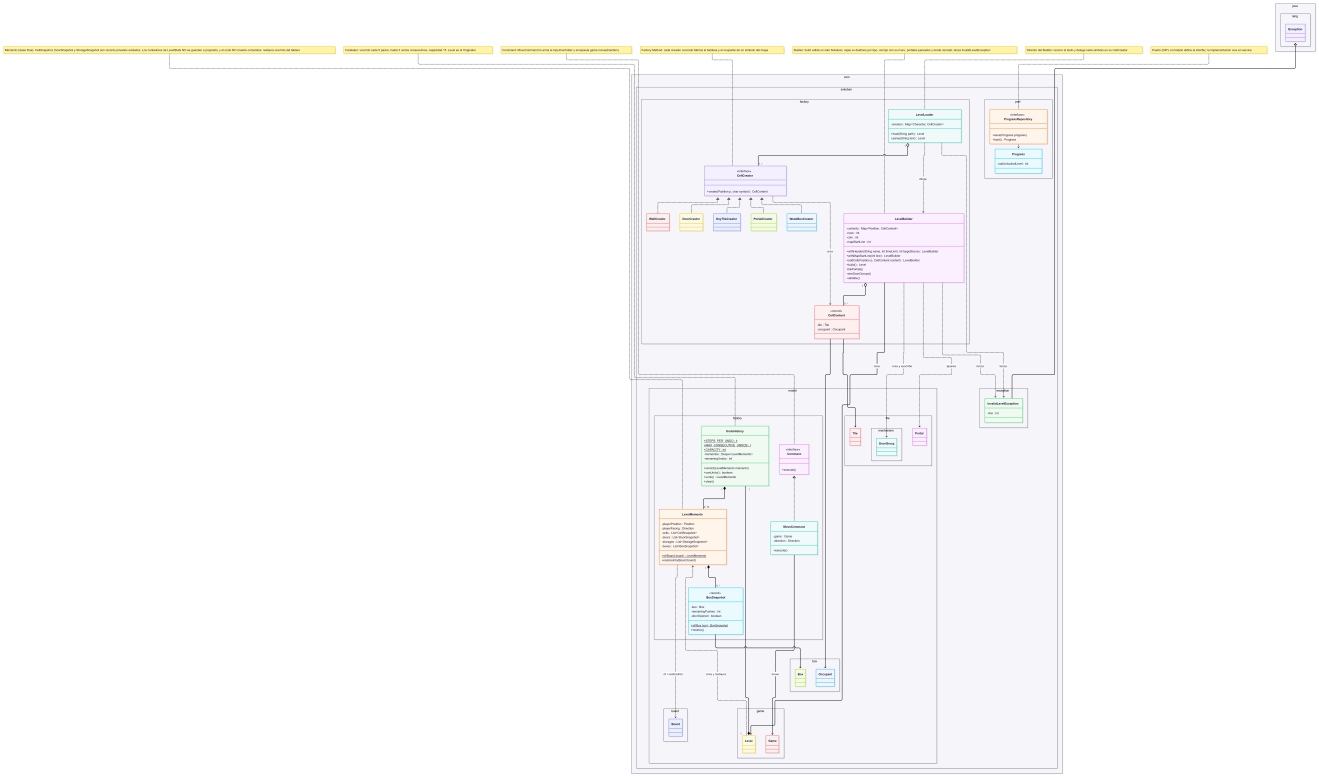
A.1 Tablero y baldosas



A.2 Ciclo de juego y estados



A.3 Historial, carga y niveles



A.4 Vista, controlador y servicios

